JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Hashtable<K,V>

java.lang.Object
  java.util.Dictionary<K,V>
    java.util.Hashtable<K,V>

Type Parameters:

K - the type of keys maintained by this map

V - the type of mapped values

All Implemented Interfaces:

Serializable, Cloneable, Map<K,V>

Direct Known Subclasses:

Properties, UIDefaults

```
public class Hashtable<K,V>
extends Dictionary<K,V>
implements Map<K,V>, Cloneable, Serializable
```

This class implements a hash table, which maps keys to values. Any non-`null` object can be used as a key or as a value.

To successfully store and retrieve objects from a hashtable, the objects used as keys must implement the `hashCode` method and the `equals` method.

An instance of `Hashtable` has two parameters that affect its performance: *initial capacity* and *load factor*. The *capacity* is the number of *buckets* in the hash table, and the *initial capacity* is simply the capacity at the time the hash table is created. Note that the hash table is *open*: in the case of a "hash collision", a single bucket stores multiple entries, which must be searched sequentially. The *load factor* is a measure of how full the hash table is allowed to get before its capacity is automatically increased. The initial capacity and load factor parameters are merely hints to the implementation. The exact details as to when and whether the rehash method is invoked are implementation-dependent.

Generally, the default load factor (.75) offers a good tradeoff between time and space costs. Higher values decrease the space overhead but increase the time cost to look up an entry (which is reflected in most `Hashtable` operations, including `get` and `put`).

The initial capacity controls a tradeoff between wasted space and the need for `rehash` operations, which are time-consuming. No `rehash` operations will *ever* occur if the initial capacity is greater than the maximum number of entries the `Hashtable` will contain divided by its load factor. However, setting the initial capacity too high can waste space.

If many entries are to be made into a `Hashtable`, creating it with a sufficiently large capacity may allow the entries to be inserted more efficiently than letting it perform automatic rehashing as needed to grow the table.

This example creates a hashtable of numbers. It uses the names of the numbers as keys:

```
Hashtable<String, Integer> numbers
  = new Hashtable<String, Integer>();
numbers.put("one", 1);
numbers.put("two", 2);
numbers.put("three", 3);
```

To retrieve a number, use the following code:

```
Integer n = numbers.get("two");
if (n != null) {
  System.out.println("two = " + n);
}
```

The iterators returned by the `iterator` method of the collections returned by all of this class's "collection view methods" are *fail-fast*: if the Hashtable is structurally modified at any time after the iterator is created, in any way except through the iterator's own `remove` method, the iterator will throw a `ConcurrentModificationException`. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future. The Enumerations returned by Hashtable's `keys` and `elements` methods are *not* fail-fast; if the Hashtable is structurally modified at any time after the enumeration is created then the results of enumerating are undefined.

Note that the fail-fast behavior of an iterator cannot be guaranteed as it is, generally speaking, impossible to make any hard guarantees in the presence of unsynchronized concurrent modification. Fail-fast iterators throw `ConcurrentModificationException` on a best-effort basis. Therefore, it would be wrong to write a program that depended on this exception for its correctness: *the fail-fast behavior of iterators should be used only to detect bugs.*

As of the Java 2 platform v1.2, this class was retrofitted to implement the `Map` interface, making it a member of the Java Collections Framework. Unlike the new collection implementations, `Hashtable` is synchronized. If a thread-safe implementation is not needed, it is recommended to use `HashMap` in place of `Hashtable`. If a thread-safe highly-concurrent implementation is desired, then it is recommended to use `ConcurrentHashMap` in place of `Hashtable`.

Since:

1.0

See Also:

Object.equals(java.lang.Object),
Object.hashCode(),
rehash(),
Collection,
Map,
HashMap,
TreeMap,
Serialized Form

Constructor Summary

Constructors

Constructor	Description
`Hashtable()`	Constructs a new, empty hashtable with a default initial capacity (11) and load factor (0.75).
`Hashtable(int initialCapacity)`	Constructs a new, empty hashtable with the specified initial capacity and default load factor (0.75).
`Hashtable(int initialCapacity, float loadFactor)`	Constructs a new, empty hashtable with the specified initial capacity and the specified load factor.
`Hashtable(Map<? extends K, ? extends V> t)`	Constructs a new hashtable with the same mappings as the given Map.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
void	`clear()`	Clears this hashtable so that it contains no keys.
Object	`clone()`	Creates a shallow copy of this hashtable.
V		

	compute(K key, BiFunction<? super K, ? super V, ? extends V> remappingFunction)	Attempts to compute a mapping for the specified key and its current mapped value, or null if there is no current mapping (optional operation).
V	**computeIfAbsent**(K key, Function<? super K, ? extends V> mappingFunction)	If the specified key is not already associated with a value (or is mapped to null), attempts to compute its value using the given mapping function and enters it into this map unless null (optional operation).
V	**computeIfPresent**(K key, BiFunction<? super K, ? super V, ? extends V> remappingFunction)	If the value for the specified key is present and non-null, attempts to compute a new mapping given the key and its current mapped value (optional operation).
boolean	**contains**(Object value)	Tests if some key maps into the specified value in this hashtable.
boolean	**containsKey**(Object key)	Tests if the specified object is a key in this hashtable.
boolean	**containsValue** (Object value)	Returns true if this hashtable maps one or more keys to this value.
Enumeration<V>	**elements**()	Returns an enumeration of the values in this hashtable.
Set<Map.Entry<K,V>>	**entrySet**()	Returns a Set view of the mappings contained in this map.

boolean	**equals**(Object o)	Compares the specified Object with this Map for equality, as per the definition in the Map interface.
V	**get**(Object key)	Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.
int	**hashCode**()	Returns the hash code value for this Map as per the definition in the Map interface.
boolean	**isEmpty**()	Tests if this hashtable maps no keys to values.
Enumeration<K>	**keys**()	Returns an enumeration of the keys in this hashtable.
Set<K>	**keySet**()	Returns a Set view of the keys contained in this map.
V	**merge**(K key, V value, BiFunction<? super V, ? super V, ? extends V> remappingFunction)	If the specified key is not already associated with a value or is associated with null, associates it with the given non-null value (optional operation).
V	**put**(K key, V value)	Maps the specified key to the specified value in this hashtable.
void	**putAll**(Map<? extends K, ? extends V> t)	Copies all of the mappings from the specified map to this hashtable.
protected void	**rehash**()	

		Increases the capacity of and internally reorganizes this hashtable, in order to accommodate and access its entries more efficiently.
V	**remove**(Object key)	Removes the key (and its corresponding value) from this hashtable.
int	**size**()	Returns the number of keys in this hashtable.
String	**toString**()	Returns a string representation of this Hashtable object in the form of a set of entries, enclosed in braces and separated by the ASCII characters " , " (comma and space).
Collection<V>	**values**()	Returns a Collection view of the values contained in this map.

Methods declared in class Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Methods declared in interface Map

forEach, getOrDefault, putIfAbsent, remove, replace, replace, replaceAll

Constructor Details

Hashtable

```
public Hashtable(int initialCapacity,
                 float loadFactor)
```

Constructs a new, empty hashtable with the specified initial capacity and the specified load factor.

Parameters:

`initialCapacity` - the initial capacity of the hashtable.

`loadFactor` - the load factor of the hashtable.

Throws:

`IllegalArgumentException` - if the initial capacity is less than zero, or if the load factor is nonpositive.

Hashtable

```
public Hashtable(int initialCapacity)
```

Constructs a new, empty hashtable with the specified initial capacity and default load factor (0.75).

Parameters:

`initialCapacity` - the initial capacity of the hashtable.

Throws:

`IllegalArgumentException` - if the initial capacity is less than zero.

Hashtable

```
public Hashtable()
```

Constructs a new, empty hashtable with a default initial capacity (11) and load factor (0.75).

Hashtable

```
public Hashtable(Map<? extends K, ? extends V> t)
```

Constructs a new hashtable with the same mappings as the given Map. The hashtable is created with an initial capacity sufficient to hold the mappings in the given Map and a default load factor (0.75).

Parameters:

`t` - the map whose mappings are to be placed in this map.

Throws:

`NullPointerException` - if the specified map is null.

Since:

1.2

Method Details

size

`public int size()`

Returns the number of keys in this hashtable.

Specified by:

`size` in interface `Map<K,V>`

Specified by:

`size` in class `Dictionary<K,V>`

Returns:

the number of keys in this hashtable.

isEmpty

`public boolean isEmpty()`

Tests if this hashtable maps no keys to values.

Specified by:

`isEmpty` in interface `Map<K,V>`

Specified by:

`isEmpty` in class `Dictionary<K,V>`

Returns:

`true` if this hashtable maps no keys to values; `false` otherwise.

keys

`public Enumeration<K> keys()`

Returns an enumeration of the keys in this hashtable. Use the Enumeration methods on the returned object to fetch the keys sequentially. If the hashtable is structurally modified while enumerating over the keys then the results of enumerating are undefined.

Specified by:

`keys` in class `Dictionary<K,V>`

Returns:

an enumeration of the keys in this hashtable.

See Also:

`Enumeration`, `elements()`, `keySet()`, `Map`

elements

`public Enumeration<V> elements()`

Returns an enumeration of the values in this hashtable. Use the Enumeration methods on the returned object to fetch the elements sequentially. If the hashtable is structurally modified while enumerating over the values then the results of enumerating are undefined.

Specified by:

`elements` in class `Dictionary<K,V>`

Returns:

an enumeration of the values in this hashtable.

See Also:

`Enumeration`, `keys()`, `values()`, `Map`

contains

`public boolean contains(Object value)`

Tests if some key maps into the specified value in this hashtable. This operation is more expensive than the `containsKey` method.

Note that this method is identical in functionality to `containsValue`, (which is part of the `Map` interface in the collections framework).

Parameters:

`value` - a value to search for

Returns:

`true` if and only if some key maps to the `value` argument in this hashtable as determined by the `equals` method; `false` otherwise.

Throws:

`NullPointerException` - if the value is `null`

containsValue

`public boolean containsValue(Object value)`

Returns true if this hashtable maps one or more keys to this value.

Note that this method is identical in functionality to `contains` (which predates the `Map` interface).

Specified by:

`containsValue` in interface `Map<K,V>`

Parameters:

`value` - value whose presence in this hashtable is to be tested

Returns:

`true` if this map maps one or more keys to the specified value

Throws:

`NullPointerException` - if the value is `null`

Since:

1.2

containsKey

```
public boolean containsKey(Object key)
```

Tests if the specified object is a key in this hashtable.

Specified by:

`containsKey` in interface `Map<K,V>`

Parameters:

`key` - possible key

Returns:

`true` if and only if the specified object is a key in this hashtable, as determined by the `equals` method; `false` otherwise.

Throws:

`NullPointerException` - if the key is `null`

See Also:

`contains(Object)`

get

```
public V get(Object key)
```

Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.

More formally, if this map contains a mapping from a key `k` to a value `v` such that `(key.equals(k))`, then this method returns `v`; otherwise it returns `null`. (There can be at most one such mapping.)

Specified by:

`get` in interface `Map<K,V>`

Specified by:

`get` in class `Dictionary<K,V>`

Parameters:

key - the key whose associated value is to be returned

Returns:

the value to which the specified key is mapped, or `null` if this map contains no mapping for the key

Throws:

`NullPointerException` - if the specified key is null

See Also:

`put(Object, Object)`

rehash

```
protected void rehash()
```

Increases the capacity of and internally reorganizes this hashtable, in order to accommodate and access its entries more efficiently. This method is called automatically when the number of keys in the hashtable exceeds this hashtable's capacity and load factor.

put

```
public V put(K key,
             V value)
```

Maps the specified `key` to the specified `value` in this hashtable. Neither the key nor the value can be `null`.

The value can be retrieved by calling the `get` method with a key that is equal to the original key.

Specified by:

`put` in interface `Map<K,V>`

Specified by:

`put` in class `Dictionary<K,V>`

Parameters:

key - the hashtable key

value - the value

Returns:

the previous value of the specified key in this hashtable, or `null` if it did not have one

Throws:

`NullPointerException` - if the key or value is `null`

See Also:

`Object.equals(Object)`, `get(Object)`

remove

```
public V remove(Object key)
```

Removes the key (and its corresponding value) from this hashtable. This method does nothing if the key is not in the hashtable.

Specified by:

`remove` in interface `Map<K,V>`

Specified by:

`remove` in class `Dictionary<K,V>`

Parameters:

`key` - the key that needs to be removed

Returns:

the value to which the key had been mapped in this hashtable, or `null` if the key did not have a mapping

Throws:

`NullPointerException` - if the key is `null`

putAll

```
public void putAll(Map<? extends K, ? extends V> t)
```

Copies all of the mappings from the specified map to this hashtable. These mappings will replace any mappings that this hashtable had for any of the keys currently in the specified map.

Specified by:

`putAll` in interface `Map<K,V>`

Parameters:

`t` - mappings to be stored in this map

Throws:

`NullPointerException` - if the specified map is null

Since:

1.2

clear

```
public void clear()
```

Clears this hashtable so that it contains no keys.

Specified by:

`clear` in interface `Map<K,V>`

clone

```
public Object clone()
```

Creates a shallow copy of this hashtable. All the structure of the hashtable itself is copied, but the keys and values are not cloned. This is a relatively expensive operation.

Overrides:

`clone` in class `Object`

Returns:

a clone of the hashtable

See Also:

`Cloneable`

toString

```
public String toString()
```

Returns a string representation of this `Hashtable` object in the form of a set of entries, enclosed in braces and separated by the ASCII characters " , " (comma and space). Each entry is rendered as the key, an equals sign =, and the associated element, where the `toString` method is used to convert the key and element to strings.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this hashtable

keySet

```
public Set<K> keySet()
```

Returns a `Set` view of the keys contained in this map. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through the iterator's own `remove` operation), the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

Specified by:

keySet in interface Map<K,V>

Returns:

a set view of the keys contained in this map

Since:

1.2

entrySet

```
public Set<Map.Entry<K,V>> entrySet()
```

Returns a Set view of the mappings contained in this map. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through the iterator's own remove operation, or through the setValue operation on a map entry returned by the iterator) the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the Iterator.remove, Set.remove, removeAll, retainAll and clear operations. It does not support the add or addAll operations.

Specified by:

entrySet in interface Map<K,V>

Returns:

a set view of the mappings contained in this map

Since:

1.2

values

```
public Collection<V> values()
```

Returns a Collection view of the values contained in this map. The collection is backed by the map, so changes to the map are reflected in the collection, and vice-versa. If the map is modified while an iteration over the collection is in progress (except through the iterator's own remove operation), the results of the iteration are undefined. The collection supports element removal, which removes the corresponding mapping from the map, via the Iterator.remove, Collection.remove, removeAll, retainAll and clear operations. It does not support the add or addAll operations.

Specified by:

values in interface Map<K,V>

Returns:

a collection view of the values contained in this map

Since:

1.2

equals

```
public boolean equals(Object o)
```

Compares the specified Object with this Map for equality, as per the definition in the Map interface.

Specified by:

`equals` in interface `Map<K,V>`

Overrides:

`equals` in class `Object`

Parameters:

o - object to be compared for equality with this hashtable

Returns:

true if the specified Object is equal to this Map

Since:

1.2

See Also:

`Map.equals(Object)`

hashCode

```
public int hashCode()
```

Returns the hash code value for this Map as per the definition in the Map interface.

Specified by:

`hashCode` in interface `Map<K,V>`

Overrides:

`hashCode` in class `Object`

Returns:

the hash code value for this map

Since:

1.2

See Also:

`Map.hashCode()`

computeIfAbsent

```
public V computeIfAbsent
(K key,
 Function<? super K, ? extends V> mappingFunction)
```

If the specified key is not already associated with a value (or is mapped to `null`), attempts to compute its value using the given mapping function and enters it into this map unless `null` (optional operation).

If the mapping function returns `null`, no mapping is recorded. If the mapping function itself throws an (unchecked) exception, the exception is rethrown, and no mapping is recorded. The most common usage is to construct a new object serving as an initial mapped value or memoized result, as in:

```
map.computeIfAbsent(key, k -> new Value(f(k)));
```

Or to implement a multi-value map, `Map<K,Collection<V>>`, supporting multiple values per key:

```
map.computeIfAbsent(key, k -> new HashSet<V>()).add(v);
```

The mapping function should not modify this map during computation.

This method will, on a best-effort basis, throw a `ConcurrentModificationException` if the mapping function modified this map during computation.

Specified by:

`computeIfAbsent` in interface `Map<K,V>`

Parameters:

`key` - key with which the specified value is to be associated

`mappingFunction` - the mapping function to compute a value

Returns:

the current (existing or computed) value associated with the specified key, or null if the computed value is null

Throws:

`ConcurrentModificationException` - if it is detected that the mapping function modified this map

computeIfPresent

```
public V computeIfPresent
(K key,
 BiFunction<? super K, ? super V, ? extends V> remappingFunction)
```

If the value for the specified key is present and non-null, attempts to compute a new mapping given the key and its current mapped value (optional operation).

If the remapping function returns `null`, the mapping is removed. If the remapping function itself throws an (unchecked) exception, the exception is rethrown, and the current mapping is left unchanged.

The remapping function should not modify this map during computation.

This method will, on a best-effort basis, throw a `ConcurrentModificationException` if the remapping function modified this map during computation.

Specified by:

`computeIfPresent` in interface `Map<K,V>`

Parameters:

`key` - key with which the specified value is to be associated

`remappingFunction` - the remapping function to compute a value

Returns:

the new value associated with the specified key, or null if none

Throws:

`ConcurrentModificationException` - if it is detected that the remapping function modified this map

compute

```
public V compute
(K key,
 BiFunction<? super K, ? super V, ? extends V> remappingFunction)
```

Attempts to compute a mapping for the specified key and its current mapped value, or `null` if there is no current mapping (optional operation). For example, to either create or append a `String` msg to a value mapping:

```
map.compute(key, (k, v) -> (v == null) ? msg : v.concat(msg))
```

(Method `merge()` is often simpler to use for such purposes.)

If the remapping function returns `null`, the mapping is removed (or remains absent if initially absent). If the remapping function itself throws an (unchecked) exception, the exception is rethrown, and the current mapping is left unchanged.

The remapping function should not modify this map during computation.

This method will, on a best-effort basis, throw a `ConcurrentModificationException` if the remapping function modified this map during computation.

Specified by:

`compute` in interface `Map<K,V>`

Parameters:

`key` - key with which the specified value is to be associated

`remappingFunction` - the remapping function to compute a value

Returns:

the new value associated with the specified key, or null if none

Throws:

`ConcurrentModificationException` - if it is detected that the remapping function modified this map

merge

```
public V merge
(K key,
 V value,
 BiFunction<? super V, ? super V, ? extends V> remappingFunction)
```

If the specified key is not already associated with a value or is associated with null, associates it with the given non-null value (optional operation). Otherwise, replaces the associated value with the results of the given remapping function, or removes if the result is `null`. This method may be of use when combining multiple mapped values for a key. For example, to either create or append a `String msg` to a value mapping:

```
map.merge(key, msg, String::concat)
```

If the remapping function returns `null`, the mapping is removed. If the remapping function itself throws an (unchecked) exception, the exception is rethrown, and the current mapping is left unchanged.

The remapping function should not modify this map during computation.

This method will, on a best-effort basis, throw a `ConcurrentModificationException` if the remapping function modified this map during computation.

Specified by:

`merge` in interface `Map<K,V>`

Parameters:

`key` - key with which the resulting value is to be associated

`value` - the non-null value to be merged with the existing value associated with the key or, if no existing value or a null value is associated with the key, to be associated with the key

`remappingFunction` - the remapping function to recompute a value if present

Returns:

the new value associated with the specified key, or null if no value is associated with the key

Throws:

`ConcurrentModificationException` - if it is detected that the remapping function modified this map

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.